



Fundamental of JAVA Programming

UNIT :- IV

Interface and Packages in JAVA



Packages

- Java two most innovative features are packages and Interfaces
- Package is the set of collection of classes and / or interfaces
- It act as a container for classes that are used.

Why Packages ?

- Limitation to reusing the classes within a program
- If we need to use classes from other programs without physically copying them into the program under development?



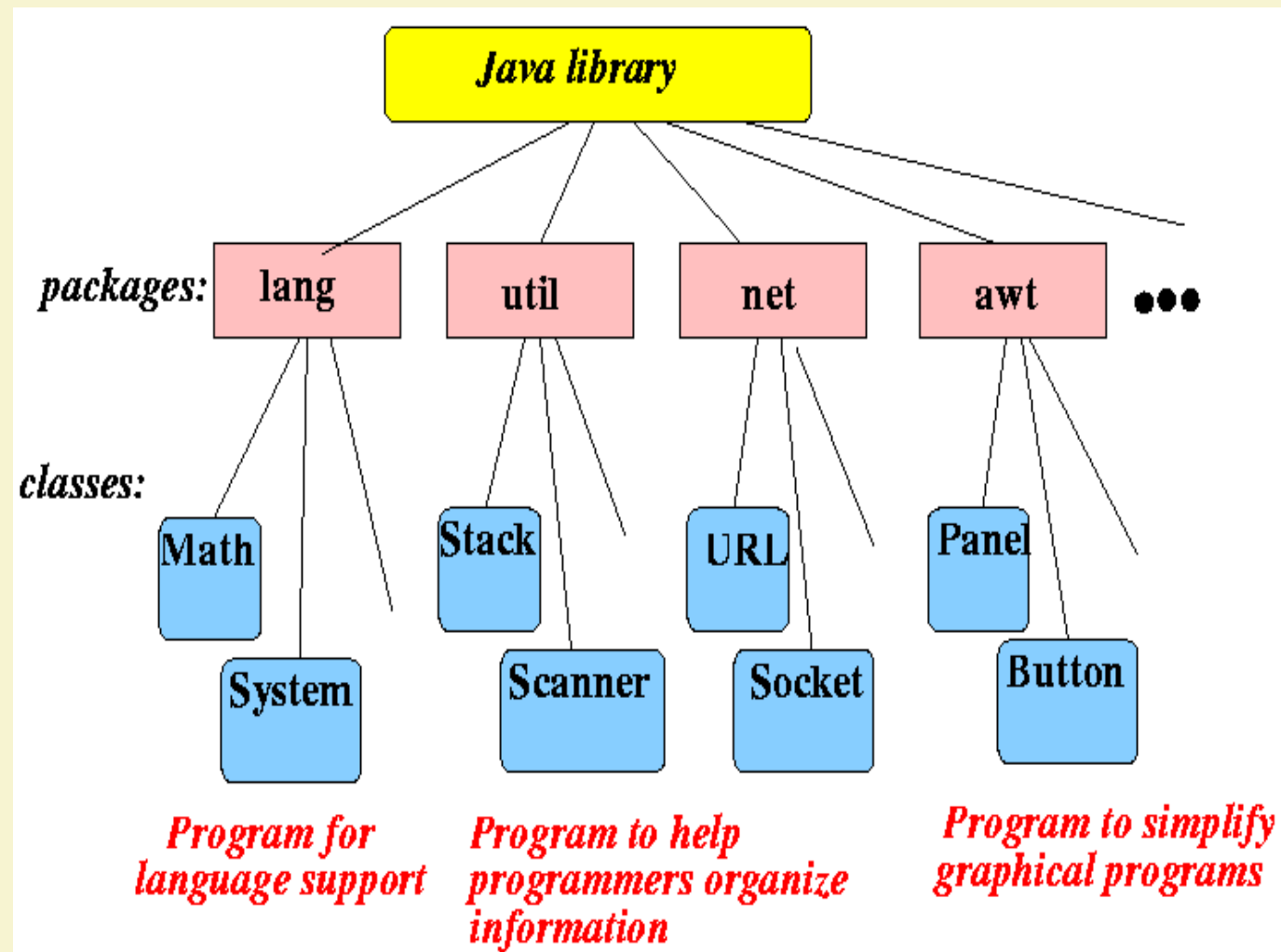
Advantages of Packages

1. **Code Reusability** :- Because package contain the group of classes (i.e Reusability and Maintainability)
2. **Resolving Name Collision** :- when multiple packages have classes with the same name
3. Package also provides the **hiding of class facility** (i.e other programs can not use the class from hidden packages)
4. **Access Limitation** can be applied with the help of package
5. **Nesting of package**



Packages

- In Java, already many predefined packages are available those are to help programmers to develop their software in any easy ways.
- E.g. java.swing ,java.lang-
- All build in package are bundle called API bundled with the JDK





Simple example of java package

```
package mypack;  
public class Simple  
{  
    public static void main(String args[])  
    {  
        System.out.println("Welcome to package");  
    }  
}
```



How to access package from another package?

- There are three ways to access the package from outside the package.

1. `import package.*;`

2. `import package.classname;`

3. fully qualified name.



Using packagename.*

File Name:-A.java

```
package pack;  
public class A  
{  
    public void msg()  
    {  
        System.out.println("Hello");  
    }  
}
```

File Name:-B.java

```
package mypack;  
import pack.*;  
class B  
{  
    public static void main(String args[])  
    {  
        A obj = new A();  
        obj.msg();  
    }  
}
```



Using packagename.classname

File Name:-A.java

```
package pack;  
public class A  
{  
    public void msg()  
    {  
        System.out.println("Hello");  
    }  
}
```

File Name:-B.java

```
package mypack;  
import pack.A;  
class B  
{  
    public static void main(String args[])  
    {  
        A obj = new A();  
        obj.msg();  
    }  
}
```




Using fully qualified name

File Name:-A.java

```
package pack;  
public class A  
{  
    public void msg()  
    {  
        System.out.println("Hello");  
    }  
}
```

File Name:-B.java

```
package mypack;  
class A  
{  
    public static void main(String args[])  
    {  
        pack.A obj = new pack.A();  
        obj.msg();  
        A obj1 = new A ();  
        obj1.display();  
    }  
    void display()  
{  
}
```



Access Modifiers in java

- There are 4 types of java access modifiers:
 - 1.private
 - 2.default
 - 3.protected
 - 4.public



Access Modifier

1) **private access modifier**

The private access modifier is accessible only within class.

2) **default access modifier**

If you don't use any modifier, it is treated as **default** by default. The default modifier is accessible only within package.

3) **protected access modifier**

The **protected access modifier** is accessible within package and outside the package but through inheritance only.

4) **public access modifier**

The **public access modifier** is accessible everywhere. It has the widest scope among all other modifiers



Access Protection

Access Modifier Access Location	Public	Protected	Default	Private
Same Class	Yes	Yes	Yes	Yes
Subclass in Same Package	Yes	Yes	Yes	No
Non Subclasses in same package	Yes	Yes	Yes	No
Subclass in Different Packages	Yes	Yes	No	No
Non Subclasses in Different packages	Yes	No	No	No



Multiple Inheritance

- Java does not support multiple inheritance
- Classes in Java can not have more than one superclass

E.g. `class A extends B extends C`

```
{  
    -----  
    -----  
}
```

- Is not permitted in Java
- Large number of real life applications require the use of multiple inheritance
- In C++ , implementation of multiple inheritance proves difficult and adds complexity to the language.
- But Java provides an alternate approach known as interfaces to support the concept of multiple inheritance



Interface

- Basically it is a kind of class
- Like classes, interface contain methods and variables but with a major difference
- Difference is that interfaces define only abstract methods and final fields
- This means that interfaces do not specify any code to implement these methods and data fields contain only constants
- Responsibility of the class that implements an interface to define the code for implementation of these methods.
- Syntax for defining an interface is very similar to that for defining a class



Syntax for Interface

- General form of an interface definition is

```
interface InterfaceName  
{  
    variable declaration;  
    method declaration;  
}
```

Interface is just keyword

- Variables are declared as

```
static final type variablename= value;
```

- Methods declaration will contain only a list of methods without any body statements

```
return type methodsname1 (parameter_list);
```

```
Interface Area  
{  
    final static float pi=3.142F;  
    float a_calculate ( float x, float y);  
    void show();  
}
```



Difference Between Class and Interface

Class	Interface
1. The members of a class can be constant or variables	1. The members of an interface are always declared as constant, i.e., their values are final
2. The class definition can contain the code for each of its methods. That is, methods can be abstracted or non-abstracted in class.	2. The methods in an interface are abstracted in nature, i.e., there is no code associated with them. It is later defined by the class that implements the interface.
3. It can be used to declare objects	3. It cannot be used to declare objects. It can only be inherited by class.
4. Use various access specifies like public, private or protected.	4. Supports only public access specifier.



Extending Interface

- As like classes, we can apply reusability concept on existing interface to create new interface.
- That is, an interface can be sub-interfaced from existing interface.
- So this existing interface is called superinterface.
- The new subinterface will inherit all the characteristics of superinterface in the manner similar to subclass.
- Syntax for this is:

```
interface name2 extends name1  
{  
    body of name2;  
}
```



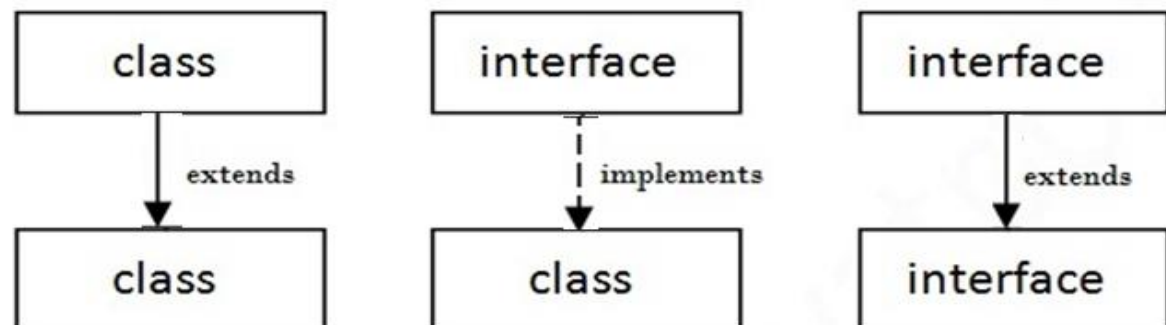
Extending Interface

```
interface Itemconstants
{
    int code=1001;
    string name="Fan";
}
interface Item extends ItemConstants
{
    void display();
}
```

```
interface Itemconstants
{
    int code=1001;
    string name="Fan";
}
interface ItemMethods extends ItemConstants
{
    void display();
}
```

Interfaces Item extends ItemConstants,ItemMethods

```
{
    -----
    -----
}
```





Implementing Interfaces

- Interfaces are used as “Superclasses” whose properties are inherited by classes.
- Therefore necessary to create a class that inherits the given interface

```
Class classname implements interfacename
```

```
{
```

```
    Body of classname
```

```
}
```

```
Class classname extends superclass
```

```
    implements interfacename1,interfacename2,.....
```

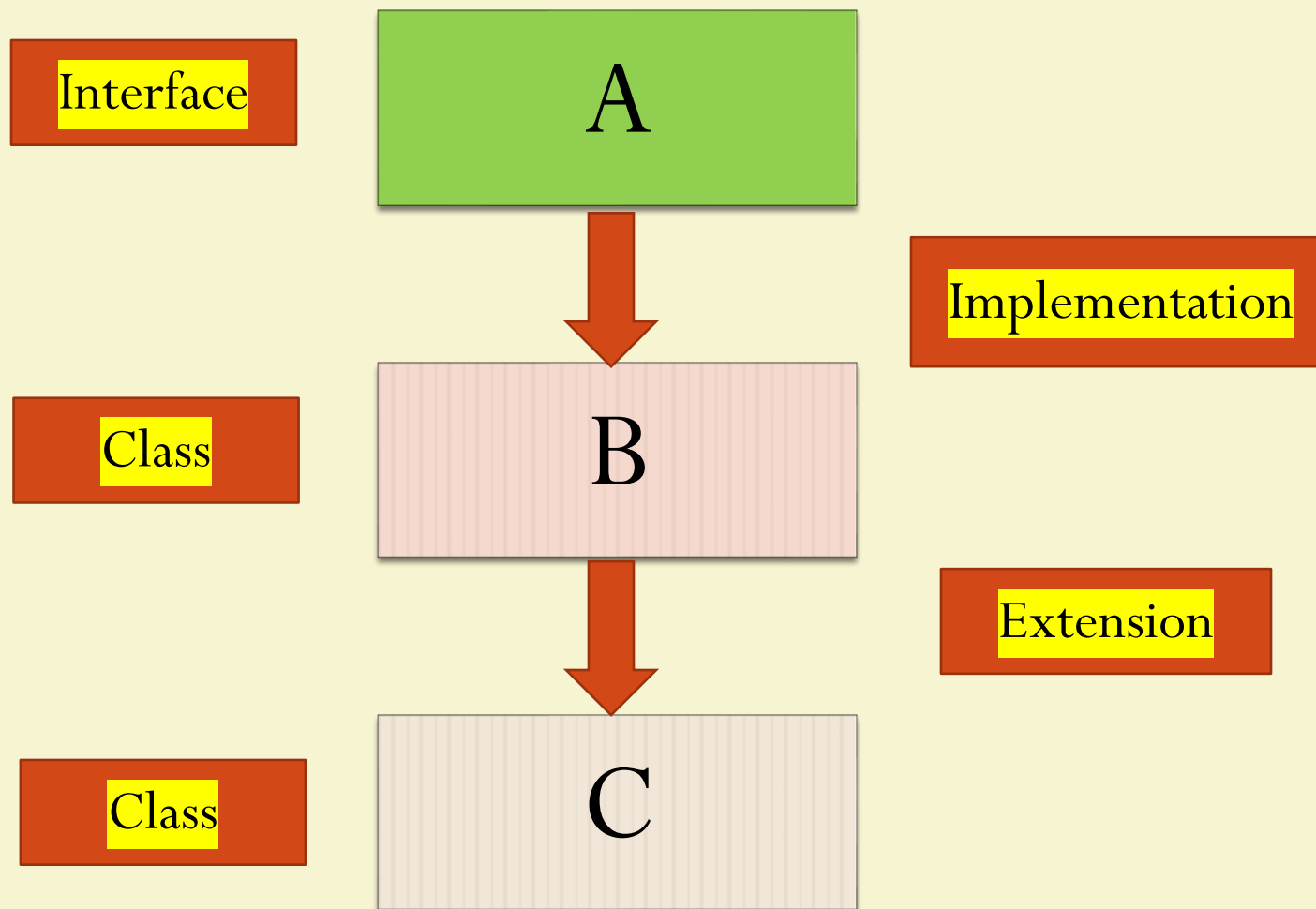
```
{
```

```
    Body of classname
```

```
}
```

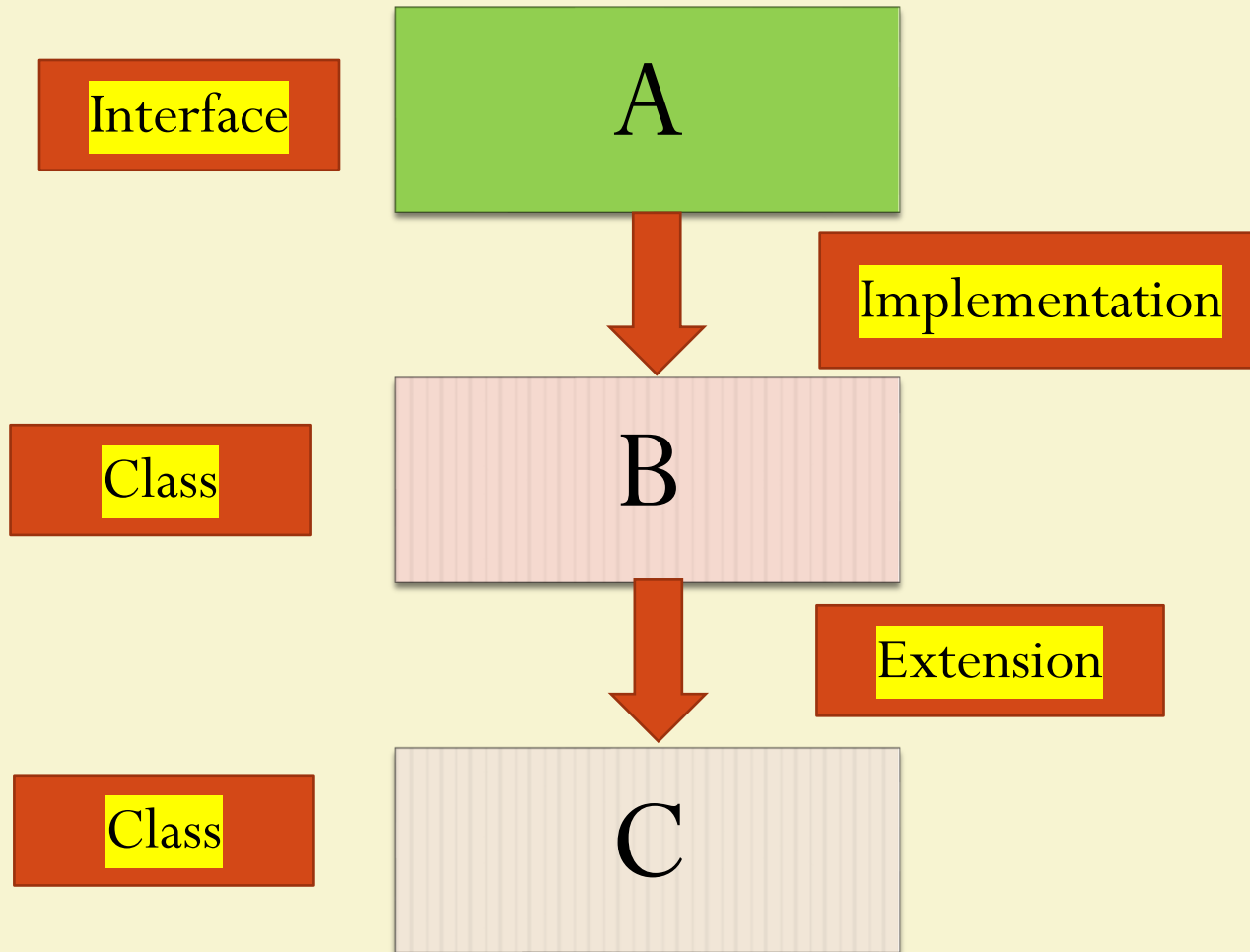


Various forms of Interface Implementation





Various forms of Interface Implementation



Default Method in Interface

```
interface Drawable
```

```
{
```

```
    void draw();
```

```
    default void msg()
```

```
{
```

```
        s.o.p("default method");
```

```
    } }
```

```
class Rectangle implements Drawable
```

```
{
```

```
    public void draw()
```

```
    {s.o.p("drawing rectangle");
```

```
    } }
```

```
class TestInterfaceDefault {
```

```
    p.s.v.m.(String args[])
```

```
    {
```

```
        Drawable d=new Rectangle();
```

```
        d.draw();
```

```
        d.msg();
```

```
    }
```

```
}
```

Static Method in Interface

```
interface Drawable
{
    void draw();
    static int cube(int x)
    {return x*x*x;}
}
```

```
class Rectangle implements Drawable
{
    public void draw()
    {
        s.o.p("drawing rectangle");
    }
}
```

```
class TestInterfaceStatic
{
    psvm(String args[])
    {
        Drawable d=new Rectangle();
        d.draw();
        s.o.p( Drawable.cube(3)
);
    }
}
```

Sr. No.	Key	Static Interface Method	Default Method
1	Basic	It is a static method which belongs to the interface only. We can write implementation of this method in interface itself	It is a method with default keyword and class can override this method
2	Method Invocation	Static method can invoke only on interface class not on class.	It can be invoked on interface as well as class
3	Method Name	Interface and implementing class , both can have static method with the same name without overriding each other.	We can override the default method in implementing class
4.	Use Case	It can be used as a utility method	It can be used to provide common functionality in all implementing classes

ABSTRACT CLASS	INTERFACE
1) Abstract class can have abstract and non-abstract methods.	Interface can have only abstract methods. Since Java 8, it can have default and static methods also.
2) Abstract class doesn't support multiple inheritance.	Interface supports multiple inheritance.
3) Abstract class can have final, non-final, static and non-static variables.	Interface has only static and final variables.
4) Abstract class can provide the implementation of interface.	Interface can't provide the implementation of abstract class.
5) The abstract keyword is used to declare abstract class.	The interface keyword is used to declare interface.
6) Example: <pre>public abstract class Shape { public abstract void draw(); }</pre>	Example: <pre>public interface Drawable { void draw(); }</pre>